Further Towards Operation POG for VDM

Nick Battle[1][0009−0001−1523−4964] and Peter Gorm Larsen[1][0000−0002−4589−1500]

Department of Electrical and Computer Engineering, Aarhus University, Denmark,
nick.battle@gmail.com, pgl@ece.au.dk

Abstract. This paper is an update to the work described in [4], regarding proof obligation generation (POG) for VDM operations. The latest work improves the approach to loop invariants, adds loop *variants*, improves POs for operation calls, including implicitly declared operations and specification statements, and adds recursive measures for operations.

1 Introduction

As described in [4], tool support for proof obligation generation (POG) from VDM *functions* is mature. However, support for the generation of obligations from VDM operations has historically been limited. This is mostly because the depth of analysis required is greater, since operations have a more complex control flow, possibly involving loops, onward operation calls, exceptions and state changes. In this paper, we describe the latest progress towards generating obligations for VDM operations.

Section 2 summarizes the progress achieved in [4]. Section 3 describes the new work for loop obligations, then Section 4 covers improvements to obligations for operation calls. Section 5 looks at the result of using the QuickCheck tool with the new obligations [3]. Section 6 considers the design of adequate constraints. Finally, Section 7 considers the remaining work.

2 Previous Status

The following principles are central to the approach described in [4], from June 2025:

- The complexity of operation control flows is managed by analysing possible paths (ignoring loops) and then creating obligations for each possible path from the start to the point of the obligation (such as a possible division by zero).
- Operations can update the values of variables along a path, but obligations are expressions which do not have state. This is represented in obligations by creating nested local scopes for updated variables, hiding the value of the same name in outer scopes.

– Operations are defined in a module context, which can include a state vector. This is represented by an extra argument in the outermost scope of obligations, effectively quantifying over all possible states. This is more complicated for VDM++ and VDM-RT, see 7.

These principles allowed the creation of some operation obligations, but the work was limited in the following regards:

– The proposed approach to loop invariants did not scale well, in particular for nested loops, and there was no support for loop variants (to prove termination).
– Operation calls from within an operation marked state variables as *ambiguous*, meaning that their values are unknown and obligations that depend on ambiguous values could not be checked.
– There was no support for recursive operation measures.
– There was no support for exception handling, or VDM++/VDM-RT states.

In this work, we present improvements to the first three points above.

3 Loop Obligations

3.1 Invariants

Loop invariants have been used for formal analysis and verification for many years [2]. An invariant is a boolean expression involving the variables that are visible to the loop, whose value is true before the loop, at the start and end of each loop iteration, and after the loop.

As introduced in [4], VDMJ provides a `@LoopInvariant` annotation. The POG uses the annotation to create POs at points before, during and after the loop annotated.

Consider the following simple loop:

```
state Sigma of
  sv : nat
end

operations
  op: nat ==> nat
  op(a) ==
  (
    dcl local : nat := 0;
    sv := a;

    -- @LoopInvariant(sv + local = a)
    while sv > 0 do
    (
      sv := sv - 1;
      local := local + 1
```

```
    );

    return a - 1
  );
```

This produces six POs:

```
Proof Obligation 1: (Unproved)
-- check invariant before while condition
(forall a:nat, mk_Sigma(sv):Sigma &
  (let local : nat = 0 in
    (let sv : nat = a in
      ((sv + local) = a))))
```

PO #1 checks the invariant before the loop starts. The context at this point has the parameter bindings for `a` and `sv`, followed by the definition of `local` and the assignment to `sv`. The obligation then adds a check of the invariant in that context on the last line.

```
Proof Obligation 2: (Unproved)
-- check invariant before first while body
(forall a:nat, mk_Sigma(sv):Sigma &
  (let local : nat = 0 in
    (let sv : nat = a in
      ((sv > 0) =>
        ((sv + local) = a)))))
```

PO #2 is very similar to #1, but with the additional condition that the while loop is actually entered, `(sv > 0) =>`. This represents the check that the invariant holds at the start of the first loop. (PO #6 covers the case where the loop is not entered.)

```
Proof Obligation 3: (Unproved)
(forall a:nat, mk_Sigma(sv):Sigma &
  (let local : nat = 0 in
    (let sv : nat = a in
      ((sv > 0) =>
        (forall local:nat, sv:nat &
          (((sv + local) = a) and (sv > 0) =>
            (sv - 1) >= 0))))))
```

PO #3 is a check within the body of the loop, since the `sv` value is a natural number and cannot be decremented below zero. Notice that the context now includes a `forall`, quantifying over the `local` and `sv` values, immediately qualified by the invariant and the loop condition. These are the two values changed by the loop, so the PO is saying "for all possible changes made by the loop, if the invariant holds and the loop has not terminated, then the obligation holds".

```
Proof Obligation 4: (Unproved)
-- check invariant preserved by while body
(forall a:nat, mk_Sigma(sv):Sigma &
  (let local : nat = 0 in
    (let sv : nat = a in
      ((sv > 0) =>
        (forall local:nat, sv:nat &
          (((sv + local) = a) and (sv > 0) =>
            (let sv : nat = (sv - 1) in
              (let local : nat = (local + 1) in
                ((sv + local) = a)))))))))
```

PO #4 follows a similar pattern to #3, which checks that if the loop is entered and the invariant holds at the start, then after performing the updates made by the body, the invariant still holds at the end. Together with PO #2, this forms the inductive basis of the loop invariant.

```
Proof Obligation 5: (Unproved)
(forall a:nat, mk_Sigma(sv):Sigma &
  (let local : nat = 0 in
    (let sv : nat = a in
      ((sv > 0) =>
        (forall local:nat, sv:nat &
          (((sv + local) = a) and (not (sv > 0)) =>
            (a - 1) >= 0))))))
```

PO #5 is caused by the `return a - 1` statement after the loop. This illustrates how the POG adds context for obligations immediately after the loop, effectively saying that we know the invariant still holds and the loop condition has not been met (i.e. the loop terminated).

```
Proof Obligation 6: (Unproved)
(forall a:nat, mk_Sigma(sv):Sigma &
  (let local : nat = 0 in
    (let sv : nat = a in
      ((not (sv > 0)) =>
        (-- Did not enter loop at 13:9
          (((sv + local) = a) =>
            (a - 1) >= 0))))))
```

The final PO #6 covers the path where the loop was not entered. In this case, the obligation still holds after the loop, even though no changes were made.

3.2 Other Loop Types

While-loops are the simplest case, but VDM also has three *for-loop* types. The POs produced follow the same overall principle as *while-loops*, with the following differences:

– *for-index* loops are of the form `for x = y to z by s do`. In this case, the PO before the loop is the case when `x = y`. If the loop was an equivalent *while-loop*, the `x` variable would be set to `y` outside, followed by a `while x <= z do` loop. In our example, the loop quantifier for `for x = 1 to a do`, would quantify over the `x` values as well as the loop state changes, as follows:

```
((1 <= a) =>
  (forall x:nat1, local:nat, sv:nat &
    (((x >= 1) and (x <= a)) and ((sv + local) = a) =>
```

– *for-set* loops are of the form `for all x in set S do`. The difficulty here is that sets are not ordered, therefore the order in which the elements are processed is not defined. To handle this, the PO uses a *ghost variable* which holds the elements processed so far; the invariant may reason about the ghost variable. By default, the ghost is called `DONE_n$`, where n is the line number of the loop[1]. The quantifier, for `for all x in {1, ..., a} do`, is as follows:

```
(({1, ..., a} <> {}) =>
 (forall x:nat1, local:nat, sv:nat, DONE_13$:set of nat1 &
    (DONE_13$ psubset {1, ..., a})
    and (x in set ({1, ..., a} \ DONE_13$))
    and ((sv + local) = a) =>
```

Then, for the obligation that checks the invariant at the end of the loop, the ghost variable is first updated to include the current `x` value before checking the invariant:

```
(let DONE_13$ : set of nat1 = DONE_13$ union {x} in
  ((sv + local) = a))
```

– *for-seq* loops are of the form `for p in S do`, where `S` is a sequence. Although sequences are ordered, the approach taken here is similar to *for-set* loops, using a ghost sequence to hold the elements done so far, allowing the invariant to reason about them. The quantifier in `for x in list do`, and the update of the ghost, is as follows (using the length of the ghost in the invariant, instead of `local`):

```
((list <> []) =>
 (forall x:nat1, DONE_13$:seq of nat1, local:nat, sv:nat &
   (DONE_13$ ^ [x] = list(1, ..., len DONE_13$ + 1))
   and ((sv + (len DONE_13$)) = a) =>
   ...
   (let DONE_13$ : seq of nat1 = DONE_13$ ^ [x] in
     ((sv + (len DONE_13$)) = a)
```

[1] The ghost can be given a more sensible name, using a second argument to the annotation.

3.3 Variants

Loop variants can be used in *while-loops* to verify that loops will terminate. This is not necessary with *for-loops*, which always terminate. Variants are introduced via the `@LoopMeasure`[2] annotation.

A loop variant is an expression whose value is a natural number, which decreases towards zero for every iteration of the loop. In the example above, the `sv` variable would act as a valid measure, so the loop would be annotated like this:

```
-- @LoopInvariant(sv + local = a)
-- @LoopMeasure(sv)
while sv > 0 do
...
```

The variant produces a single obligation, saying that the measure (held in a variable called `LOOP_n$`) at the start of a loop iteration is greater than the measure at the end:

```
Proof Obligation 1: (Unproved)
-- check measure after each while body
(forall a:nat, mk_Sigma(sv):Sigma &
  (let local : nat = 0 in
    (let sv : nat = a in
      (forall local:nat, sv:nat &
        (((sv + local) = a) and (sv > 0) =>
          (let LOOP_13$ : nat = sv in
            (let sv : nat = (sv - 1) in
              (let local : nat = (local + 1) in
                sv < LOOP_13$))))))))
```

3.4 Runtime Checks

Loop invariants and loop measures are both checked by the VDMJ interpreter at runtime, so even if proof obligations are not formally discharged, it may be possible to find invariant/measure violations via traditional test methods. Ghost variables appear in the runtime context as variables.

[2] Because of the similarity to function measures. Using @LoopVariant could be confused with @LoopInvariant.

4 Operation Calls

4.1 Simple Calls

Simple operation calls do not return a value. These cause a problem in proof obligations because a called operation can make arbitrary updates to state data, which must be represented in the PO context. But unlike (say) a direct assignment to state, we cannot easily calculate the state updates that an operation will make.

The approach taken in [4] was to naively calculate the variable update set, and mark these as *ambiguous*, meaning that their values are unknown after the call. If the PO subsequently reasoned about these values, it was marked as "UNCHECKED".

The new approach calculates a more accurate transitive closure of the variables that can be updated by an operation call. The possible values of these variables are then quantified over, before being qualified by any pre/post constraints on the operation being called. For example:

```
state Sigma of
    sv : nat
end

operations
    op: nat ==> nat
    op(x) == (
        op2(x + 1);
        return 1/sv    -- Proof obligation here! (sv <> 0)
    );

    op2(a:nat)         -- NOTE: no return value
    ext wr sv          -- Changes "sv"
    pre a > sv
    post sv > a;


Proof Obligation 1: (Unproved)
(forall x:nat, mk_Sigma(sv):Sigma &
  pre_op2((x + 1), mk_Sigma(sv)))

Proof Obligation 2: (Unproved)
(forall x:nat, mk_Sigma(sv):Sigma &
  (let $oldState = mk_Sigma(sv) in
    (forall sv:nat &     -- Because op2 has "ext wr sv"
      pre_op2((x + 1), $oldState)
      and post_op2((x + 1), $oldState, mk_Sigma(sv)) =>
        sv <> 0)))       -- PO for the return 1/sv
```

The first PO checks that the precondition for op2 is always met. The second PO records the "old" state, before quantifying over possible values of the affected

state, sv. This is then qualified by both the precondition over the old state, and the postcondition over the old state plus the new updated state. If all of that context is met, then we are reasoning about a valid scenario and so the final obligation must hold in that case.

The pattern of quantifying over the possibilities, then qualifying them with the constraints, is similar to the loop invariant checks, where we quantify over the possible state updates in the loop, and qualify them with the loop invariant and loop condition.

This approach also allows POs to be generated for implicit operation definitions (as the example above) or for specification statements, which are similar but inline.

Note that an operation call may be to another module, which has its own opaque state. That may then call back to the original module, and update its state. In general, we are only concerned about the possible state updates in the local module, but we do have to allow a remote module to have an arbitrary state too. For example, if we call an operation in module "B", with opaque state "B`Gamma", from local module "A" with state "Sigma", the POs look like this[3]:

```
Proof Obligation 1: (Unproved)
(forall x:nat, mk_Sigma(sv):Sigma &
  (forall $oldState:B`Gamma &
    B`pre_op2((x + 1), $oldState)))

Proof Obligation 2: (Unproved)
(forall x:nat, mk_Sigma(sv):Sigma &
  (forall $oldState:B`Gamma, $newState:B`Gamma &
    (-- Change local state here too? eg. forall sv:nat &
      B`pre_op2((x + 1), $oldState)
      and B`post_op2((x + 1), $oldState, $newState) =>
        sv <> 0)))
```

Note that the PO knows nothing about Gamma values and does not try to track them. But it quantifies over the possible Gamma states when calling operations in the B module. Therefore the onus is on the specifier to write strong pre/post conditions for the operation, so that the valid Gamma states are identifiable. This is discussed in Section 6.

4.2 Return Values

As well as updating state, operation calls can return values which are used in calculations by the caller. The ISO VDM-SL Standard [1] limits the places where operations can be called to yield a value, since the order of operation evaluation is significant, and the order of calls in an arbitrary expression is not defined. However, most VDM tools relax this constraint in anticipation of VDM++/RT, which use operation calls with less rigour.

[3] POs have global visibility of all modules' state names, though remote state structures remain opaque.

The problem for the POG is firstly to find and extract the operation calls from an expression. It must then work out the order of evaluation of those operation calls, before adding the possible results to the values quantified over as a result of making each call. The possible results must then be substituted back into the original expression to continue with the context path.

For example, if a specification makes two calls *to the same operation* and adds the results together:

```
def rv = op2(x + 1) + op2(x * 2) in return 1/rv
```

Then the important obligation looks like this:

```
Proof Obligation 1: (Unproved)
(forall x:nat, mk_Sigma(sv):Sigma &
  (let $oldState = mk_Sigma(sv) in
    (forall sv:nat, $op2:nat &       -- Possible sv + result
       pre_op2((x + 1), $oldState)
       and post_op2(
         (x + 1), $op2, $oldState, mk_Sigma(sv)) =>
      (let $oldState = mk_Sigma(sv) in
        (forall sv:nat, $op2$1:nat & -- Possible sv + result
           pre_op2((x * 2), $oldState)
           and post_op2(
             (x * 2), $op2$1, $oldState, mk_Sigma(sv)) =>
          (let rv = ($op2 + $op2$1) in  -- Substituted back
            rv <> 0))))))
```

The `$op2` and `$op2$1` variables are the return values from the two calls to `op2`. This naming pattern extends to an arbitrary number of calls to the same operation in an expression. Return values are quantified over in the PO context, along with the possible updates to `sv`, qualified by the operation's precondition and postcondition.

Note that the extracted calls to `op2` are performed in the order shown. This is because VDMJ performs the addition in left-to-right order. In general, extracted operation calls have to appear in tool evaluation order in the PO context.

Finally, for every valid combination of results and state updates, the results are added together to set `rv` before the final obligation check.

This approach does permit most operation calls with return values to be handled by the POG, but some cases are not possible, such as when an expression generates an unknown number of calls, like { `op2(a) | a in set S` } – here we cannot statically determine how many calls are made by the set comprehension, and the resulting POs are marked as "UNCHECKED".

4.3 Recursive Measures

Operations may be recursive in VDM, similar to functions, though operations do not have to terminate[4]. Recursive functions can define a `measure` to verify

[4] For example, a `while true do` loop does not terminate

that recursion will terminate. We enable a similar feature for operations via an `@OperationMeasure` annotation. For example[5]:

```
state Sigma of
    sv : nat
end

operations
    -- @OperationMeasure(a)
    fac: nat ==> nat
    fac(a) ==
        if a = 1
        then return 1
        else return a * fac(a-1);

Proof Obligation 1: (Unproved)
(forall a:nat, mk_Sigma(sv):Sigma &
  (let MEASURE_6$ : nat = a in
    (not (a = 1) =>
      MEASURE_6$ > measure_fac(a - 1, mk_Sigma(sv)))))
```

The presence of the `@OperationMeasure` annotation causes the creation of a `measure_fac` function, which has the same signature as `pre_fac`, but which evaluates the measure expression provided.

The obligations generated are very similar to those for recursive function calls, but with the addition of the state argument. The temporary variable holding the measure includes the line number of the operation.

This direct recursive example is a special case of a more general pattern where mutually recursive cycles of operation calls are required to define a measure which decreases at each step of the cycle. For example, if the factorial example is divided into two operations, in two modules, which call each other alternately, the obligations would be as follows:

```
Proof Obligation 1: (Unproved)
(forall a:nat, mk_Sigma(sv):Sigma &
  (let MEASURE_11$ : nat = a in
    (not (a = 1) =>
      (forall $oldState:B`Gamma &
        MEASURE_11$ > B`measure_fac(a - 1, $oldState)))))

Proof Obligation 2: (Unproved)
(forall b:nat, mk_Gamma(xv):Gamma &
  (let MEASURE_28$ : nat = b in
    (not (b = 1) =>
      (forall $oldState:A`Sigma &
        MEASURE_28$ > A`measure_fac(b - 1, $oldState)))))
```

[5] The subtle error in this operation is picked up in Section 5.

Note that the state of the remote modules is opaque to each caller, so a quantifier is added to allow the state to potentially affect the measure. In the direct recursive case, the PO can construct the current Sigma state directly.

4.4 Runtime Checks

As with loop invariants, operation measures are checked by the VDMJ interpreter at runtime, so even if proof obligations are not formally discharged, it may be possible to find measure violations via traditional test methods. Ghost variables appear in the runtime context as variables.

5 QuickCheck

The new obligations produced by this work are checkable by the QuickCheck tool [3], which complements VDMJ's POG. For example, the recursive factorial operation in Section 4.3 produces the following results:

```
> qc
PO #1, FAILED in 0.009s
Counterexample:
fac: recursive operation obligation at line 11:25
(forall a:nat, mk_Sigma(sv):Sigma &
 --> sv = 0, a = 0
  (let MEASURE_7$ : nat = a in
   --> MEASURE_7$ = 0
    (not (a = 1) =>
      MEASURE_7$ > measure_fac(a - 1, mk_Sigma(sv)))))
       --> Error 4065: Value -1 is not a nat at line 4:20

PO #2, FAILED in 0.001s
Counterexample:
fac: subtype obligation at line 11:30
(forall a:nat, mk_Sigma(sv):Sigma &
 --> sv = 0, a = 0
  (let MEASURE_7$ : nat = a in
    (not (a = 1) =>
      (a - 1) >= 0)))
      --> returns false
>
```

In this particular example, the operation fails if the argument passed is zero, because the exit of the recursion check is `a = 1` rather than `a <= 1`. This can be corrected by changing the test or adding a precondition.

The new obligations created by this work reduce the number of "UNCHECKED" results that the POG produces, allowing QuickCheck to analyse more of them. As the work has progressed, the POG has been repeatedly tested on a large corpus of VDM-SL specifications that are distributed with the tool, producing over 7000 obligations. In [4], the POG marked 9.6% of these as "UNCHECKED"; with the latest work, that figure is down to 2.4%.

6 Adequate Constraints

The new obligations, for both loops and operation calls, depend on invariants and postconditions to qualify the large number of possibilities, representing all of the state changes that could occur. The ability to discharge obligations which follow loops or operation calls therefore depends on the *adequacy* of those constraints: they must provide enough information to enable a theorem prover to discharge the obligations.

With no constraints, loops and operations could make any changes to state, limited only by the static analysis of which values they affect. This may permit value combinations which violate obligations (cause a counterexample) but which the specifier knows *cannot happen*. In this case, constraints have to be added or tightened to eliminate those combinations.

On the other hand, an over-constrained specification may eliminate valid combinations which are required for the discharge of (typically existential) obligations that follow them. In this case, the constraints must be relaxed.

Judging how to balance the strength of constraints can be a challenge, even for experienced users. However, the need to provide enough information to fulfil obligations is a useful guide. It is our belief that tool support aimed at helping to design constraints, in the context of the obligations they affect, would be very productive.

7 Future Work

Noting the limitations in Section 2, the current work has addressed loop invariants and operation calls, including measures. The remaining work items from Section 7, "Future Work" of [4] are:

- Statements that handle exceptions (`always`, `trap` and `tixe`) cause control flows that are currently too complex to handle.
- Specifications that include variable hiding can easily confuse the POG and produce invalid POs.
- The correct operation of the POG itself must be determined. This should ultimately be linked to a proof theory for VDM operations.
- The complex state and control flows of VDM++ and VDM-RT cause a host of problems that have yet to be solved.

In addition, as noted in 4.2, we cannot yet deal with operation calls in expressions that cannot be statically determined.

As mentioned in Section 6, it is our belief that useful progress can be made by considering tool support for the creation of model constraints, given the context of the proof obligations that need to be discharged under those constraints.

Acknowledgements We are grateful for the support of the European Union, Aarhus University, Newcastle University and the Grundfos Foundation.

References

1. Information technology – programming languages, their environments and system software interfaces – vienna development method – specification language – part 1: Base language (Dec 1996), international standard specifying the VDM-SL formal specification language
2. Hoare, C.A.R.: An axiomatic basis for computer programming. Commun. ACM **12**(10), 576–580 (Oct 1969). https://doi.org/10.1145/363235.363259, https://doi.org/10.1145/363235.363259
3. N.Battle, M.Ellyton: QuickCheck for VDM. In: Proceedings of the 22nd Overture Workshop (2024). https://doi.org/10.48550/arXiv.2410.02046
4. N.Battle, P.G.Larsen: Towards Operation Proof Obligation Generation in VDM. In: Proceedings of the 23rd Overture Workshop (2025). https://doi.org/10.48550/arXiv.2506.12858